



Cifrado Alberti

Facultad de Ingeniería

Criptografía

Dr. Alfonso Francisco de Abiega l'Eglise

2027-1

Grupo: 3

Ayala Hernández María Fernanda

López Hernández Miriam Amisadai

Tepal Briseño Hansel Yael

Ugartechea González Luis Antonio

Fecha de entrega: 10 de septiembre de 2026

1 Explicación del Cifrado Alberti

1.1 Descripción General

El programa implementa una variante del cifrado de rueda de Alberti utilizando dos alfabetos: una rueda externa fija y una rueda interna definida mediante una clave. La posición de la rueda interna puede cambiar durante el procesamiento del mensaje, lo que provoca que la sustitución de las letras no permanezca constante durante todo el archivo.

A diferencia de un cifrado por desplazamiento fijo, el programa permite definir una posición inicial, un periodo de rotación y una dirección de giro. De esta forma, después de procesar cierta cantidad de letras, la rueda interna cambia de posición y se modifica la correspondencia utilizada para las siguientes letras.

1.2 Parámetros de Ejecución

El programa recibe la información necesaria mediante argumentos de línea de comandos. La ejecución general sigue la siguiente forma:

```
1 ./cifradoAlberti <c|d> <entrada> <salida> <posicion> <periodo> <+|->  
    <clave>
```

Listing 1: Forma general de ejecución del programa.

Los parámetros permiten seleccionar si se desea cifrar o descifrar, indicar el archivo de entrada y el archivo de salida, establecer la posición inicial de la rueda, determinar cada cuántas letras debe realizarse un giro, seleccionar la dirección del giro y proporcionar el alfabeto utilizado como clave.

1.3 Validación de la Clave

La rueda interna se representa mediante una clave formada por las 26 letras mayúsculas del alfabeto inglés. Para que la clave sea válida, debe contener exactamente 26 caracteres, utilizar únicamente letras de la A a la Z y no repetir ninguna letra.

De forma simplificada, la validación realizada por el programa puede representarse de la siguiente manera:

```
1 si longitud(clave) != 26:  
2     rechazar clave  
3  
4 para cada letra de la clave:  
5     si no pertenece a A-Z:  
6         rechazar clave  
7     si la letra ya fue utilizada:
```

```
8         rechazar clave
9
10    aceptar clave
```

Listing 2: Lógica general para validar la clave de la rueda interna.

Esta comprobación garantiza que la clave represente una permutación completa del alfabeto y que cada símbolo de la rueda interna aparezca una sola vez.

1.4 Proceso de Cifrado

Para cifrar una letra se obtiene primero su posición dentro del alfabeto externo. Posteriormente se suma la posición actual de la rueda interna y se aplica módulo 26 para mantener el resultado dentro de los límites del alfabeto.

La operación puede expresarse conceptualmente como:

```
1 indice = posicion_de_la_letra
2 indice_rueda = (indice + posicion_actual) % 26
3 letra_cifrada = rueda_interna[indice_rueda]
```

Listing 3: Representación simplificada del cifrado de una letra.

La letra cifrada se obtiene de la rueda interna definida por la clave. El programa conserva el uso de mayúsculas y minúsculas, mientras que los caracteres que no son letras se escriben sin modificación.

1.5 Proceso de Descifrado

El descifrado realiza el procedimiento inverso. Primero se localiza la letra cifrada dentro de la rueda interna y después se elimina el desplazamiento producido por la posición actual de la rueda.

```
1 indice_interno = buscar(letra_cifrada, rueda_interna)
2 indice_original = (indice_interno - posicion_actual + 26) % 26
3 letra_original = alfabeto[indice_original]
```

Listing 4: Representación simplificada del proceso de descifrado.

La suma de 26 antes de aplicar el módulo evita obtener posiciones negativas. Para recuperar correctamente el mensaje es necesario utilizar los mismos parámetros empleados durante el cifrado.

1.6 Rotación de la Rueda

Una característica importante de esta implementación es que la rueda interna no permanece necesariamente en la misma posición. El programa cuenta únicamente las letras procesadas y, cuando se alcanza el periodo especificado, modifica la posición de la rueda.

```
1 contador_letras++
2
3 si contador_letras % periodo == 0:
4     si direccion == '+':
5         posicion_actual = (posicion_actual + 1) % 26
6     si direccion == '-':
7         posicion_actual = (posicion_actual - 1 + 26) % 26
```

Listing 5: Lógica general del giro periódico de la rueda.

Los espacios, números, signos de puntuación y saltos de línea no incrementan este contador, ya que el giro se calcula solamente con base en las letras procesadas.

1.7 Lectura y Escritura de Archivos

El programa trabaja con un archivo de entrada y genera un archivo de salida diferente. Cada carácter se lee de manera secuencial; si corresponde a una letra, se procesa con la posición actual de la rueda y, en caso contrario, se copia directamente al archivo de salida.

```
1 abrir archivo de entrada
2 crear archivo de salida
3
4 mientras exista un caracter:
5     leer caracter
6     transformar caracter si es una letra
7     escribir resultado en el archivo de salida
8     actualizar la rueda cuando corresponda
9
10 cerrar archivos
```

Listing 6: Flujo general de procesamiento del archivo.

También se comprueba que el archivo de entrada y el archivo de salida tengan nombres diferentes y se validan posibles errores de apertura, escritura, lectura y cierre.

1.8 Ejemplo de Ejecución

Un ejemplo de cifrado utilizando una posición inicial de 3, un periodo de 4 letras, giro hacia atrás y el alfabeto invertido como clave es:

```
1 ./cifradoAlberti c mensaje_original.txt mensaje_cifrado.txt 3 4 -
  ZYXWVUTSRQPONMLKJIHGFEDCBA
```

Listing 7: Ejemplo de ejecución para cifrar un archivo.

Para descifrar el archivo deben conservarse exactamente la misma posición, periodo, dirección y clave:

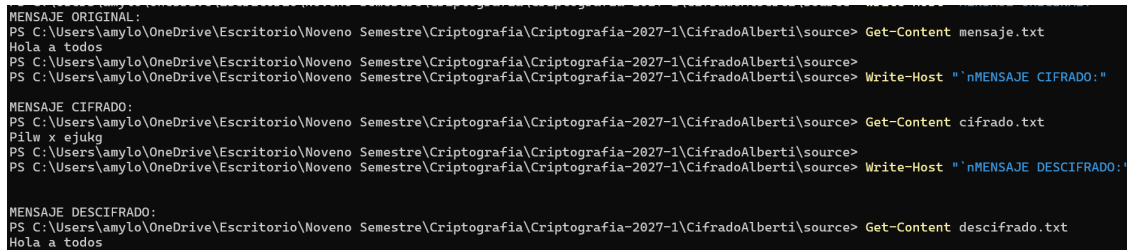
```
1 ./cifradoAlberti d mensaje_cifrado.txt mensaje_recuperado.txt 3 4 -  
ZYXWVUTSRQPONMLKJIHGFEDCBA
```

Listing 8: Ejemplo de ejecución para descifrar un archivo.

Al finalizar, el contenido de `mensaje_recuperado.txt` debe coincidir con el contenido del archivo original.

1.9 Salida en Terminal

A continuación se muestra una prueba de ejecución del programa, en la cual se observa el mensaje original, el mensaje cifrado y posteriormente el mensaje recuperado mediante el proceso de descifrado.



```
 Mensaje Original:  
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\Criptografia\Criptografia-2027-1\CifradoAlberti\source> Get-Content mensaje.txt  
Hola a todos  
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\Criptografia\Criptografia-2027-1\CifradoAlberti\source>  
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\Criptografia\Criptografia-2027-1\CifradoAlberti\source> Write-Host "`nMensaje Cifrado:"  
  
Mensaje Cifrado:  
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\Criptografia\Criptografia-2027-1\CifradoAlberti\source> Get-Content cifrado.txt  
P!lw x ejukg  
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\Criptografia\Criptografia-2027-1\CifradoAlberti\source>  
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\Criptografia\Criptografia-2027-1\CifradoAlberti\source> Write-Host "`nMensaje Descifrado:"  
  
Mensaje Descifrado:  
PS C:\Users\amylo\OneDrive\Escritorio\Noveno Semestre\Criptografia\Criptografia-2027-1\CifradoAlberti\source> Get-Content descifrado.txt  
Hola a todos
```

Figure 1: Ejemplo de ejecución del Cifrado Alberti mostrando el mensaje original, cifrado y descifrado.

1.10 Repositorio y Código Fuente

Link del repositorio: <https://github.com/HansMarcus01/Criptografia-2027-1>

Código fuente: <https://github.com/HansMarcus01/Criptografia-2027-1/blob/feat-cifrado-alberti/cifradoAlberti/source/cifradoAlberti.c>